

École Nationale Supérieure Polytechnique de Yaoundé

Département de Génie Informatique

ANI-IA 4068 : Coder pour la VR, AR, XR II

RAPPORT DES TP_s DES SEMAINES 1 - 4

Nom : MEZAGO Wilfried Aymar

Matricule : 22P001

Spécialité : AIA 4

Sous la supervision de :

Ing TEUGUIA TADJUIDJE Rodolf Sédéris

Année académique 2025–2026

Table des matières

Introduction	1
1 SEMAINE 1 : IEEE 754 Arithmétique Flottante	2
1.1 La fonction InspectFloat	2
1.2 Kahan et Welford	3
2 SEMAINE 2 : Géométrie Vectorielle ($Vec2d$, $Vec3d$, $Vec4d$)	4
2.1 Vec2d complet	4
2.2 Vec3d avec Gram-Schmidt	4
2.3 Vec4d et projection perspective simple	4
3 SEMAINE 3 : Matrices et Visualisation	5
3.1 Mat4d et inverse	5
3.2 Projections et Rendu LookAt	5
4 SEMAINE 4 : Rotations et Quaternions ($Quat$)	7
4.1 Quaternions complets	7
4.2 Animation SLERP	7
5 SEMAINE 5 : Décomposition en Valeurs Singulières (SVD) et Géométrie Projective	9
5.1 Validation de l'algorithme SVD 3x3 et Pseudo-Inverse	9
5.2 Perspective de Résolution d'Homographie	10
6 SEMAINE 6 : Pipeline de Traitement d'Image Linéaire et Algorithmes Rapides	11
6.1 Manipulation et primitives de structures de pixels	11
6.2 Convolutions spatiales (Flou Gaussien et Filtres de Sobel)	11
6.3 Image Intégrale et Seuillage Adaptatif de Bradley	12
Conclusion	13

Introduction

Ce projet s'inscrit dans le cadre de l'implémentation complète d'un système de Réalité Augmentée (RA) sur flux vidéo, régi par une contrainte : **l'interdiction absolue d'utiliser des dépendances externes (telles qu'OpenCV, Eigen, glm, OpenGL ou DirectX) pour le cœur de calcul. L'intégralité du code devant reposer sur des concepts fondamentaux réimplémentés en C++ natif.**

L'objectif ici est de franchir, étape par étape, la chaîne complète de la RA :

- **Semaines 1 à 4 (Fondations)** : Conception d'un moteur d'algèbre linéaire (NKMath) immunisé contre la dérive numérique, capable de gérer les transformations spatiales (matrices TRS, quaternions, interpolations SLERP) et de simuler informatiquement une caméra (fonctions LookAt et de projection perspective) à travers un rasteriseur logiciel rudimentaire.
- **Semaines 5 et 6 (Vision par Ordinateur)** : Transition vers l'analyse de flux et la géométrie projective. Cette phase exploite le moteur géométrique pour traiter des matrices de données complexes (décomposition en valeurs singulières ou SVD) afin de résoudre les équations d'homographie, et met en place un pipeline d'analyse d'image bas niveau performant (convolutions spatiales, filtrages de Sobel, construction d'images intégrales et seuillage adaptatif) destiné à détecter des marqueurs de type ArUco.

Chapitre 1

SEMAINE 1 : IEEE 754

Arithmétique Flottante

L'objectif de cette semaine était d'apprendre la représentation et la manipulation des nombres flottants : coordonnées 3D, matrices de projection, résidus de calibration, valeurs de pixels avec **IEEE 754**.

1.1 La fonction InspectFloat

Il était question dans ce TP d'implémenter la fonction `inspectFloat(float x)` qui affiche les 3 champs (signe, exposant, mantisse) en binaire sur la console. Les résultats obtenus ont permis d'aboutir aux réponses suivantes :

1. **Pourquoi 0.1f n'est-il pas exact ?** Le résultat obtenu laisse remarquer que la mantisse présente un motif périodique infini (1100). Comme la mémoire d'un float est limitée à 23 bits pour la mantisse, ce motif est tronqué. La valeur réelle stockée n'est donc pas exactement 0.1, mais une approximation.
2. Après implémentation, nous avons bien **Signe : 0**, **Exposant brut : 01111111** (soit 127 en décimal), et **Mantisse : 00000000000000000000000**. Donc 1.0f a une représentation exacte, sans perte de précision.
3. Les résultats permettent d'identifier **+Inf** en 0 - 11111111 - 000000000000000000000000 .
4. De même, pour **nan** on obtient 1 - 11111111 - 100000000000000000000000 .
5. Avec **-0.0f** on a 1 - 00000000 - 000000000000000000000000 et **0.0f** nous donne 0 - 00000000 - 000000000000000000000000.
6. L'affichage de **1.17549e-38** nous donne 0 - 00000001 - 000000000000000000000000.

1.2 Kahan et Welford

Il était question ici de démontrez empiriquement les problèmes de précision.

1. La comparaison d'une boucle d'accumulation classique avec l'algorithme de Kahan révèle que cette dernière donne un resultat bien plus précis.
2. La mise en évidence d'une formule de variance naive avec celle de Welford montre que la méthode de Welford reste stable et juste.
3. Les deux valeurs de Epsilon machine sont exactement sont identique soit 1.19209e-07

Chapitre 2

SEMAINE 2 : Géométrie Vectorielle (*Vec2d*, *Vec3d*, *Vec4d*)

L'objectif de cette semaine était la compréhension des vecteurs qui sont la base du pipeline AR.

2.1 Vec2d complet

Ici nous avons procédé à l'implémentation de *Vec2d* , son produit scalaire, son produit vectoriel simulé en 2D.

Produit scalaire :

$$\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y + u_z v_z = \|\vec{u}\| \|\vec{v}\| \cos(\theta)$$

2.2 Vec3d avec Gram-Schmidt

Nous avons implémenté les calculs de directions et de normales, ainsi que le procédé de Gram-Schmidt capable de reconstruire une base orthonormée propre à partir de 3 vecteurs quelconques non coplanaires.

Projection d'un vecteur $\vec{v}$ sur $\vec{u}$ (Gram-Schmidt) :

$$\text{proj}_{\vec{u}}(\vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|^2} \vec{u}$$

2.3 Vec4d et projection perspective simple

Nous avons implémenté des vecteurs à 4 composantes en différenciant les points (poids $w = 1$) et les directions ($w = 0$) afin de séparer correctement l'impact des translations lors des projections.

Chapitre 3

SEMAINE 3 : Matrices et Visualisation

Cette semaine avait pour objectif la compréhension des matrices et leur visualisation.

3.1 Mat4d et inverse

On a implémenté l'inversion d'une matrice 4×4 construite par l'algorithme de Gauss-Jordan avec recherche de pivot partiel pour préserver la stabilité numérique. On vérifiait l'intégrité du calcul en s'assurant que :

$$M \times M^{-1} = I$$

(Où I est la matrice identité).

3.2 Projections et Rendu LookAt

Nous avons ici procédé par :

- l'élaboration de matrices de translation, de mise à l'échelle et de rotation autour d'un axe arbitraire (formule de Rodrigues)

$$\mathbf{R} = \mathbf{I} + (\sin \theta)\mathbf{K} + (1 - \cos \theta)\mathbf{K}^2$$

(Où $\mathbf{K}$ est la matrice antisymétrique du produit vectoriel de l'axe).

- Fusion des transformations dans l'ordre strict TRS
- Implémentation de la fonction LookAt simulant le positionnement d'un observateur et d'une matrice de projection perspective gérant le découpage géométrique et export du rendu filaire d'un cube dans un fichier d'image portable

(PPM puis conversion en png).

Chapitre 4

SEMAINE 4 : Rotations et Quaternions (*Quat*)

Cette semaine a pour objectif de montrer le rôle crucial que jouent les Quaternions dans le pipeline AR tout en mettant le problème majeur qu'ils résolvent.

4.1 Quaternions complets

Nous avons implémenté et vérifié des fonctions de création de quaternions à partir d'un axe et d'un angle (*FromAxisAngle*), ainsi que de la rotation d'un vecteur 3D par un quaternion unitaire (*Rotate*), montrant qu'une rotation de $\pi/2$ d'un vecteur initialement placé sur l'axe X autour de l'axe Y renvoie fidèlement un vecteur dirigé vers l'axe -Z (à un epsilon près). Nous avons également réalisé des conversions automatiques entre quaternions et matrices de rotation 3×3 à l'aide de 50 générations de quaternions aléatoires normalisés.

4.2 Animation SLERP

Ici, nous avons effectué un calcul itératif et rendu d'une rotation de cube sur 60 frames reliant une orientation nulle à un demi-tour (π) autour de l'axe Y selon la formule

$$\text{SLERP}(q_1, q_2, t) = \frac{\sin((1-t)\theta)}{\sin \theta} q_1 + \frac{\sin(t\theta)}{\sin \theta} q_2$$

. Résultat, le code a produit une séquence d'images PPM numérotées décrivant un mouvement à vitesse angulaire parfaitement constante, sans subir le blocage de cardan (Gimbal Lock) des angles d'Euler. En comparaison avec une série parallèle de 60 frames d'animation en utilisant l'interpolation linéaire classique des composantes des quaternions **LERP** il en est ressorti que bien que les images s'exportent de la même façon, la trajectoire linéaire ne préserve pas naturellement la vitesse angulaire

(accélération puis décélération au centre de l'arc de cercle), mettant en valeur la supériorité esthétique et cinématique du SLERP pour l'interpolation d'orientations.

Les images se trouvent au chemin : . dans les deux formats.

Chapitre 5

SEMAINE 5 : Décomposition en Valeurs Singulières (SVD) et Géométrie Projective

Cette a marqué l'introduction d'outils d'algèbre avancée indispensables à l'estimation de pose et à la calibration à partir de correspondances de points 2D/3D. Il était principalement question ici de saisir l'importance de la compréhension de la SVD.

5.1 Validation de l'algorithme SVD 3x3 et Pseudo-Inverse

La décomposition en valeurs singulières (SVD) factorise une matrice A sous la forme $U\Sigma V^T$. Notre implémentation valide trois aspects mathématiques cruciaux :

- **Reconstruction et Orthogonalité** : La vérification que $U \cdot \Sigma \cdot V^T = A$ à une tolérance près de 10^{-6} , et l'assurance que U et V sont strictement orthogonales ($U \cdot U^T = I$).
- **Gestion du Rang Déficient** : Le test sur une matrice de rang 1 (lignes colinéaires) démontre que l'algorithme isole correctement les directions nulles en renvoyant une dernière valeur singulière $\sigma_z \approx 0$.
- **Résolution des Moindres Carrés via la Pseudo-Inverse (A^+)** : Face à un système surdéterminé $Ax = b$ (plus d'équations que d'inconnues), la pseudo-inverse calculée par la SVD permet de trouver le vecteur x qui minimise l'erreur quadratique $\|Ax - b\|$. Les tests unitaires valident la convergence de l'erreur résiduelle sous le seuil de 10^{-5} .

5.2 Perspective de Résolution d'Homographie

L'architecture mise en place prépare la résolution de la matrice d'homographie H (matrice 3×3). La SVD de cette matrice d'équations permet d'extraire la solution optimale (le vecteur propre associé à la plus petite valeur singulière), ouvrant la voie à la projection de textures virtuelles sur le marqueur plan.

Chapitre 6

SEMAINE 6 : Pipeline de Traitement d'Image Linéaire et Algorithmes Rapides

Cette dernière semaine concrétise la lecture des pixels et la manipulation de structures de données d'images (*NkImage*) pour isoler les motifs géométriques des marqueurs ArUco.

6.1 Manipulation et primitives de structures de pixels

La structure *NkImage* permet l'allocation, la modification de pixels en mémoire (*SetPixelRGBA*) et l'écriture au format brut PPM (*SavePPM*). Les tests valident la génération de dégradés mathématiques et le dessin de primitives géométriques (lignes, rectangles, cercles) par manipulation directe du tampon mémoire (buffer).

6.2 Convolutions spatiales (Flou Gaussien et Filtres de Sobel)

Pour extraire les contours du marqueur ArUco présent dans l'image d'entrée (*input_aruco.ppm*), un pipeline classique de traitement du signal est implémenté :

- **Réduction du bruit** : Application d'un noyau Gaussien 5×5 normalisé (somme des coefficients égale à 1) pour l'élimination des hautes fréquences parasites.
- **Calcul de gradients** : Application des masques de Sobel 3×3 en X (horizontal) et Y (vertical) pour mettre en évidence les sauts d'intensité.
- **Magnitude** : Fusion des deux images de gradients par la fonction *Combine-Gradient* ($\sqrt{G_x^2 + G_y^2}$). L'instrumentation temporelle intégrée au code mesure

l'efficacité de ces convolutions imbriquées sur l'image entière.

6.3 Image Intégrale et Seuillage Adaptatif de Bradley

La binarisation (conversion en noir et blanc strict) est une étape critique sous conditions d'éclairage variable.

- **Algorithme d'Image Intégrale (SAT - Summed-Area Table)** : Une structure `IntegralImage` est générée à partir du niveau de gris. Chaque cellule (x, y) contient la somme de tous les pixels situés au-dessus et à gauche. Le code évalue précisément le temps de construction de cette table en millisecondes.
- **Seuillage Adaptatif (Bradley-Roth)** : Grâce à l'image intégrale, le calcul de la moyenne des pixels d'un voisinage de taille variable (31×31) s'exécute en temps constant $\mathcal{O}(1)$ (seulement 4 accès mémoire par pixel au lieu de $31 \times 31 = 961$ accès). Le pixel est binarisé à noir si sa valeur est inférieure à un certain pourcentage (seuil de 7%) de la moyenne locale. Cela permet d'isoler parfaitement les carrés noirs et blancs du marqueur ArUco, indépendamment des ombres portées.

Conclusion

L'intégration et la validation des différents modules de tests au cours de ces six semaines marquent l'achèvement d'un moteur de traitement et de simulation complet, conçu de manière totalement autonome et sans aucune dépendance tierce. Ce parcours a permis de transformer des concepts théoriques exigeants en briques logicielles robustes et interconnectées : les deux premières semaines ont posé les bases de la rigueur numérique à travers la maîtrise de la norme IEEE 754 (somme de Kahan, variance de Welford) et l'élaboration d'une bibliothèque d'algèbre vectorielle alignée en mémoire ; les semaines 3 et 4 ont concrétisé la manipulation spatiale en introduisant les matrices de transformation TRS, le calcul de caméras virtuelles (*LookAt*, projections perspectives) et la gestion fluide des rotations par les quaternions (SLERP) afin d'aboutir à un premier rasteriseur logiciel capable de projeter et d'animer des objets 3D ; enfin, les semaines 5 et 6 ont propulsé ce système vers la vision par ordinateur en combinant la décomposition en valeurs singulières (SVD) pour la résolution de systèmes complexes au moindres carrés avec un pipeline complet d'analyse d'image (convolutions de Sobel/Gauss, image intégrale et seuillage adaptatif de Bradley).